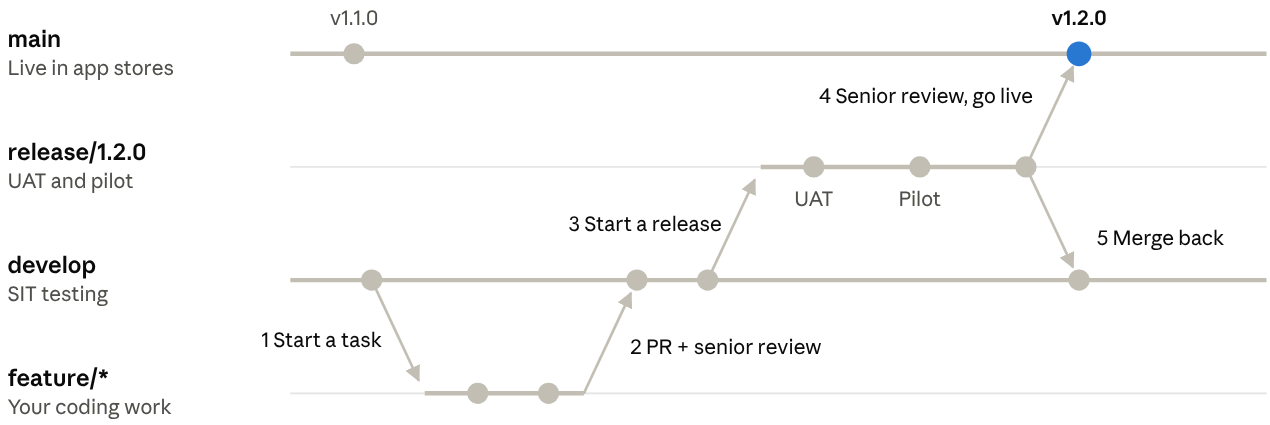
# Team Git Workflow: From Code to Post

Every change starts on its own branch. It reaches the app stores only after a senior review, SIT, UAT and a pilot.

**How code moves from your branch to the app store**




branch flow · feature to develop to release to main

Following the numbers: developers code on a feature branch, merge it into `develop` for SIT, then a release branch goes through UAT and pilot before going live on `main`.

The branch names here are a suggestion.

## Branching

There are five kinds of branch. Only `main` and `develop` stay forever.

| Branch | What it holds | Starts from | Goes into | Tested in |
|---|---|---|---|---|
| `main` | The live app | Always there | Nothing | Production |
| `develop` | Finished, reviewed features | Always there | A release branch | SIT |
| `feature/*` | One task | `develop` | `develop` | Your simulator or emulator |

| Branch | What it holds | Starts from | Goes into | Tested in |
|---|---|---|---|---|
| `release/*` | One version getting ready to ship; bug fixes only | `develop` | `main`, then back into `develop` | UAT, then pilot |
| `hotfix/*` | An urgent fix for the live app | `main` | `main` and `develop` | Quick UAT, then production |

Nobody pushes straight to `main` or `develop`. Every change goes in through a reviewed pull request.

**Working with branches**

```
# See all branches (* marks the one you are on)
git branch -a

# Make a new branch from develop and switch to it
git checkout develop
git pull origin develop
git checkout -b feature/own-account-list

# Switch to another branch
git checkout develop

# Delete a branch after it has been merged
git branch -d feature/own-account-list
```

One task = one branch. Keep branches short-lived, ideally merged within a few days, so they don't drift far from `develop`.

## Naming conventions

Names tell the team what a branch or commit is for without opening it.

**Branch names:** `type/what-it-does`, all lowercase, words joined with hyphens. Add the ticket number if there is one.

| Type | Example |
|---|---|
| Feature | `feature/transfer-confirmation-screen` |

| Type | Example |
| --- | --- |
| Feature with ticket | `feature/MB-142-own-account-list` |
| Bug fix | `fix/balance-not-refreshing` |
| Release | `release/1.2.0` |
| Hotfix | `hotfix/1.2.1-login-crash` |

**Commit messages:** `type: what changed`, short and in the present tense.

| Type | Use it for | Example |
| --- | --- | --- |
| `feat` | New feature | `feat: add transfer confirmation screen` |
| `fix` | Bug fix | `fix: show error when balance fails to load` |
| `refactor` | Tidying code without changing what it does | `refactor: move account loading into repository` |
| `test` | Tests | `test: cover empty account list` |
| `docs` | Documentation | `docs: add setup steps to README` |
| `chore` | Build settings and libraries | `chore: update Kotlin version` |

**Version numbers:** `MAJOR.MINOR.PATCH`, for example `1.2.0`. New features raise the middle number, fixes raise the last one, and big breaking changes raise the first. Every store upload also needs a higher build number (iOS) or `versionCode` (Android).

## Commit, push and pull

Commit saves your work on your computer, push sends it to GitHub, and pull brings the team's work down to you.

| Command | What it does | When to do it |
| --- | --- | --- |
| `git add` | Picks the changes to include in the next commit | Before every commit |

| Command | What it does | When to do it |
|---|---|---|
| `git commit` | Saves a snapshot of your changes on your computer, with a message | Each time one small piece works |
| `git push` | Uploads your commits to GitHub so the team can see them | At least at the end of each day, and before opening a PR |
| `git pull` | Downloads the latest commits from GitHub and merges them into your branch | Before starting work, and before pushing |
| `git fetch` | Downloads the latest commits without merging them yet | When you want to look before you merge |

```
# Commit
git status
git add .
git commit -m "feat: add own account list"

# Push (use -u the first time for a new branch)
git push -u origin feature/own-account-list
git push

# Pull the latest develop into your branch
git pull origin develop
```

**Good habits to do:**

- Commit small and often. One commit = one idea.
- Pull before you push, so you find conflicts on your computer, not on GitHub.
- Never commit passwords, keys or build files. Check with `git status` first.

**If you get a merge conflict**

1. Git marks the clashing lines with `<<<<<<<`, `=======` and `>>>>>>>`.
2. Open the file, keep the right code and delete the markers.
3. Build and test the app again.
4. Run `git add .` then `git commit` to finish the merge, and push.
5. Not sure which code is right? Ask the person who wrote the other change.

# Day-to-day developer workflow

Every task follows the same eight steps.

1. **Update:** get the latest `develop`.
2. **Branch:** make a new branch for your task.
3. **Commit:** save your work in small steps with clear messages.
4. **Keep up to date:** pull in new `develop` changes at least once a day.
5. **Push:** upload your branch to GitHub.
6. **Open a pull request** into `develop` and ask a senior to review it.
7. **Fix review comments** and push again. The pull request updates by itself.
8. **Merge and tidy up:** once approved, merge and delete the branch.

```
# 1. Update
git checkout develop
git pull origin develop

# 2. Branch
git checkout -b feature/transfer-confirmation-screen

# 3. Commit
git add .
git commit -m "feat: add transfer confirmation screen"

# 4. Keep up to date
git fetch origin
git merge origin/develop

# 5. Push
git push -u origin feature/transfer-confirmation-screen

# 6. Open the pull request on GitHub (base: develop)

# 7. Fix review comments, then push again
git add .
git commit -m "fix: handle empty amount"
git push
```

```
# 8. After merging on GitHub
git checkout develop
git pull origin develop
git branch -d feature/transfer-confirmation-screen
```

Before each commit, run `git status` and `git diff`. Make sure no passwords, keys or test account details are included.

## Merge requests (pull requests) and code review

A pull request (PR) can only be merged after a senior approves it and the automatic checks pass. You can't approve your own PR.

GitHub calls it a pull request; GitLab calls the same thing a merge request.

**How a merge request works**

1. **Open it:** on GitHub, choose your branch and set the base to `develop`.
2. **Add reviewers:** at least one senior developer.
3. **Checks run:** the app is built and tests run automatically.
4. **Review:** the reviewer leaves comments, then approves or asks for changes.
5. **Update:** you fix the comments and push. The PR updates by itself.
6. **Merge:** once approved and checks pass, click Merge, then delete the branch.

**How to merge**

| Merging | Method | Why |
|---|---|---|
| `feature/*` into `develop` | Squash and merge | Turns all your small commits into one clean commit |
| `release/*` into `main` | Merge commit | Keeps the full release history |
| `main` back into `develop` | Merge commit | Brings release and hotfix fixes back |

If GitHub says the PR has conflicts, pull the latest `develop` into your branch, fix the conflicts on your computer (see Commit, push and pull), and push again.

**When you open a PR**

- Keep it to one task, so it is easy to review.

- Fill in the template below. Add screenshots for any screen changes.
- Test it on a simulator or emulator first, and say what you tested.
- Reply to every comment, with a fix or a reason.

**When you review a PR**

- Review within one working day.
- Comment on the code, not the person, and suggest fixes.
- Only approve code you would be happy to see in production.

**PR template to follow:**

```
## What this changes
Short description. Ticket: MB-142

## How I tested it
- [ ] Works on iOS simulator / Android emulator
- [ ] Tested no network, empty data and wrong input
- [ ] Other screens still work

## Screenshots
Before and after, for screen changes

## Security checklist
- [ ] No passwords, keys or secrets in the code
- [ ] Sensitive data saved only in Keychain / Android Keystore
- [ ] No account numbers or personal data in logs
- [ ] All network calls use HTTPS
```

**What reviewers might look for**

- **It works:** does what the ticket asks, including errors and edge cases.
- **It's secure:** passes the security checklist above.
- **It fits the structure:** follows the team's pattern, such as MVVM.
- **It's easy to read:** clear names, short functions, no leftover code.
- **It's tested:** new logic has tests, and old tests still pass.

## From SIT to production

Each of six steps matches one Git step. A senior reviews the code before every move to UAT, pilot and production.

| ACLEDA step | What happens in Git | Branch | Where it runs | Must pass before moving on |
|---|---|---|---|---|
| Requirement received | Create a branch named after the task | None yet | Nowhere yet | Scope agreed |
| Development | Code, then merge by PR | `feature/*` into `develop` | Your simulator or emulator | PR approved |
| SIT | Build from `develop` | `develop` | SIT environment | SIT tests pass |
| Review, then UAT | Start a release branch | `release/*` | UAT environment | Senior review, UAT sign-off |
| Review, then Pilot | Tag a test version, for example `v1.2.0-rc1` | `release/*` | Small pilot group, for example TestFlight or a Google Play closed test | Senior review, no pilot problems |
| Review, then Production | Merge into `main`, tag `v1.2.0`, merge back into `develop` | `main` | App Store and Google Play | Senior review, change approval |

Once a release branch starts, only bug fixes go into it. New features wait in `develop` for the next release.

```
# Start a release for UAT
git checkout develop
git pull origin develop
git checkout -b release/1.2.0
git push -u origin release/1.2.0

# Tag a test version for the pilot
git tag -a v1.2.0-rc1 -m "Pilot version 1"
git push origin v1.2.0-rc1

# After the PR into main is merged, tag the live version
git checkout main
git pull origin main
git tag -a v1.2.0 -m "Release 1.2.0"
git push origin v1.2.0
```